

Introduction

In this assignment, we studied three types of knowledge representation: First Order Logic (FOL), Conjunctive Normal Form (CNF), and Bayesian Networks. We applied various proof algorithms, mostly resolution-refutation, to answer questions about logical systems represented in these forms. This often required converting clauses from FOL to CNF before applying resolution-refutation. The final part of the assignment focused on Bayesian Networks and joint probabilities to answer questions about probabilistic logical systems.

Part 1. Propositional Logic (Knights and Knaves)

Q1-1: On the Island of Knights and Knaves, you meet two people. A says, "B is a knave." B says, "Neither A nor I are knaves." Who is what?

A says "B is a Knave" : $\text{Says}(A, \text{Knave}(B))$
B says "Neither A or B are Knaves" : $\neg \text{Knave}(A) \wedge \neg \text{Knave}(B) : \text{Knight}(A) \wedge \text{Knight}(B)$

Suppose A is a knave: $\text{Knave}(A)$
 $\text{Knave}(A) \wedge \text{Says}(A, \text{Knave}(B)) \Rightarrow \neg \text{Knave}(B)$
 $\neg \text{Knave}(B) \Rightarrow \text{Knight}(B)$
 $\text{Knight}(B) \wedge \text{Says}(B, \neg \text{Knave}(A)) \Rightarrow \underline{\neg \text{Knave}(A)}$

This is a Contradiction, So our Assumption A was a knave is wrong. it must be the case $\neg \text{Knave}(A) \Rightarrow \text{Knight}(A)$

Since $\text{Knight}(A)$:
 $\text{Knight}(A) \wedge \text{Says}(A, \text{Knave}(B)) \Rightarrow \text{Knave}(B)$

Therefore A is a knight and B is a knave

Q1-2:

A says "We are the same kind" : $\text{Says}(A, \text{same}(A, B))$

B says "We are different kinds" : $\text{Says}(B, \neg \text{same}(A, B))$

suppose A is a knight : $\text{Knight}(A)$

$\text{Knight}(A) \wedge \text{Says}(A, \text{same}(A, B)) \Rightarrow \text{same}(A, B)$

$\text{Knight}(A) \wedge \text{same}(A, B) \Rightarrow \text{Knight}(B)$

$\text{Knight}(B) \wedge \text{Says}(B, \neg \text{same}(A, B)) \Rightarrow \neg \text{same}(A, B)$

$\text{Knight}(B) \wedge \neg \text{same}(A, B) \Rightarrow \text{Knave}(A)$

This is a contradiction and our Assumption $\text{Knight}(A)$ must be false

$\neg \text{Knight}(A) \Rightarrow \text{Knave}(A)$

Since A is a knave : $\text{Knave}(A)$

$\text{Knave}(A) \wedge \text{Says}(A, \text{same}(A, B)) \Rightarrow \neg \text{same}(A, B)$

$\text{Knave}(A) \wedge \neg \text{same}(A, B) \Rightarrow \neg \text{Knave}(B)$

$\neg \text{Knave}(B) \Rightarrow \text{Knight}(B)$

Therefore A is a knave and B is a knight

Part 2. Propositional Logic (Proof by Resolution Refutation)

Q2-1:

#2

KB1: $\text{Attended}(\text{heather}, \text{Meeting}) \vee \neg \text{Invite}(\text{heather}, \text{Meeting})$

KB2: $\text{Want}(\text{heather}, \text{Meeting}) \Rightarrow \text{Invite}(\text{heather}, \text{Meeting})$

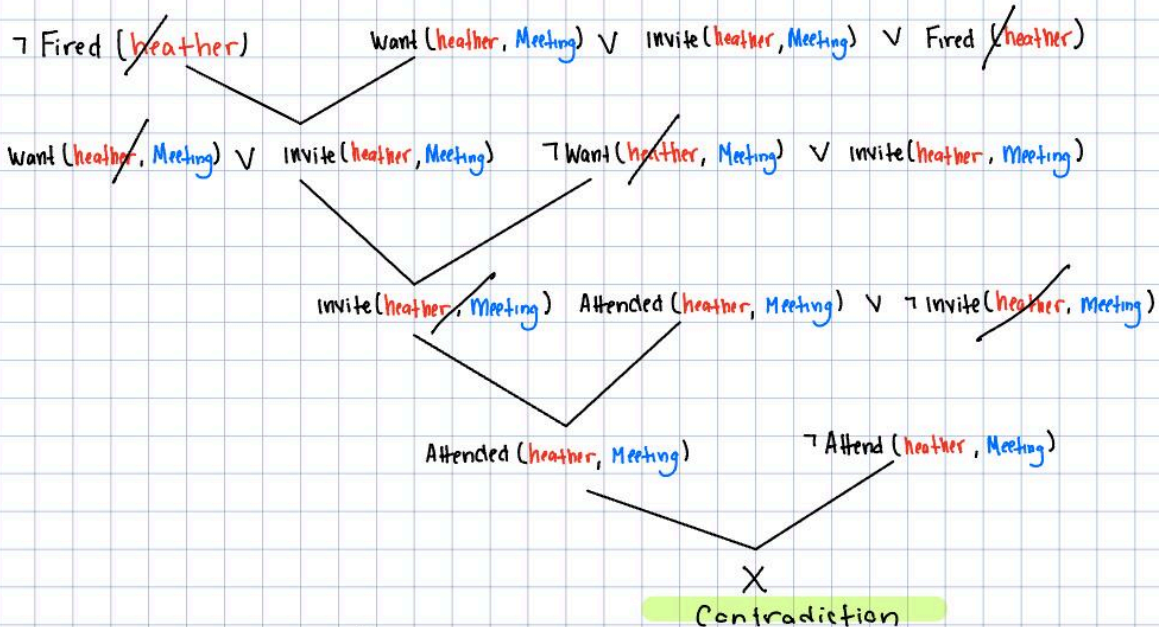
KB3: $\neg \text{Attend}(\text{heather}, \text{Meeting})$

KB4: $\neg \text{Want}(\text{heather}, \text{Meeting}) \wedge \neg \text{Invite}(\text{heather}, \text{Meeting}) \Rightarrow \text{Fired}(\text{heather})$

KB2 converted: $\neg \text{Want}(\text{heather}, \text{Meeting}) \vee \text{Invite}(\text{heather}, \text{Meeting})$

KB4 converted: $\neg(\neg \text{Want}(\text{heather}, \text{Meeting}) \wedge \neg \text{Invite}(\text{heather}, \text{Meeting})) \vee \text{Fired}(\text{heather})$

$\text{Want}(\text{heather}, \text{Meeting}) \vee \text{Invite}(\text{heather}, \text{Meeting}) \vee \text{Fired}(\text{heather})$



Therefore $\text{KB} \wedge \neg \text{Fired}(\text{heather})$ is unsatisfiable
and $\text{KB} \models \text{Fired}(\text{heather})$

Part 3. First Order Predicate Logic

Q3-1

#3

- 1) $\exists x (Student(x) \wedge teacher(x))$
- 2) $\forall x \exists y (Student(x) \Rightarrow (test(y) \wedge \neg passed(x, y)))$
- 3) $\exists y \forall x \forall z (Test(y) \wedge Student(x) \Rightarrow passed(x, y) \wedge Test(z) \wedge Student(y) \Rightarrow passed(x, z) \Rightarrow z = y)$

Q3-2:

- 1) $Student(a) \wedge teacher(a)$ Skolemization
- 2) $\forall x \exists y (\neg Student(x) \vee (test(y) \wedge \neg passed(x, y)))$ replace $\Rightarrow$
 $\forall x (\neg Student(x) \vee test(sk(x)) \wedge \neg passed(x, sk(x)))$ Skolemization
 $\neg Student(x) \vee test(sk(x)) \wedge \neg passed(x, sk(x))$ Drop $\forall$
 $(\neg Student(x) \vee Test(sk(x))) \wedge (\neg Student(x) \vee passed(x, sk(x)))$ Distribute $\wedge$
- 3) $\exists y \forall x \forall z ((Test(y) \wedge Student(x)) \Rightarrow passed(x, y)) \wedge ((Test(z) \wedge Student(y)) \Rightarrow passed(z, y) \Rightarrow z = y)$ replace $\Rightarrow$
 $\exists y \forall x \forall z (\neg (Test(y) \wedge Student(x)) \vee passed(x, y)) \wedge (\neg (Test(z) \wedge Student(y)) \vee passed(z, y)) \vee z = y)$
Move negation in $\neg (\neg Test \vee \neg Student \vee passed) \vee z = y$
 $\exists y \forall x \forall z (\neg Test(y) \vee Student(x) \vee passed(x, y)) \wedge (\neg Test(z) \wedge Student(y) \wedge \neg passed(z, y) \vee z = y)$
Skolemization
 $\forall x \forall z (\neg Test(a) \vee Student(x) \vee passed(a, x)) \wedge (\neg Test(z) \wedge Student(y) \wedge \neg passed(z, y) \vee z = y)$
Drop quantifiers
Distribute $\vee$
 $\neg Test(a) \vee Student(x) \vee passed(a, x) \wedge (\neg Test(z) \wedge z = y) \wedge (Student(y) \vee z = y) \wedge (\neg passed \vee z = y)$

Part 4. First Order Logic (Proof by Resolution Refutation)

Q4-1:

#4

1. Each creature must be exactly one of the following types: Ogre, Dwarf, or Goblin.

$$\forall x [\text{creature}(x) \Rightarrow (\text{ogre}(x) \vee \text{Dwarf}(x) \vee \text{Goblin}(x))]$$

2. Some creatures are monsters.
3. Monsters can eat some of the other creatures.

$$\exists x [\text{creature}(x) \wedge \text{Monster}(x)]$$

3. Monsters can eat some of the other creatures.

$$\forall x [\text{Monsters}(x) \Rightarrow \exists y (\text{creature}(y) \wedge \text{Eats}(x, y))]$$

4. Creatures can defeat any creatures they eat.

$$\forall x \forall y [\text{creature}(x) \wedge \text{Eats}(x, y) \Rightarrow \text{defeat}(x, y)]$$

5. Defeating is transitive. If x defeats creature y, and y defeats z, then x defeat z.

$$\forall x \forall y \forall z [\text{defeat}(x, y) \wedge \text{defeat}(y, z) \Rightarrow \text{defeat}(x, z)]$$

6. If a creature is an Ogre, then that creature can eat dwarves.

$$\forall x \forall y [\text{ogre}(x) \wedge \text{dwarf}(y) \Rightarrow \text{eat}(x, y)]$$

7. If a creature is a Dwarf, then that creature can defeat goblins.

$$\forall x \forall y [\text{dwarf}(x) \wedge \text{goblin}(y) \Rightarrow \text{defeat}(x, y)]$$

8. All goblins are monsters.

$$\forall x [\text{goblin}(x) \Rightarrow \text{Monster}(x)]$$

Q4-2:

#4

1. Each creature must be exactly one of the following types: Ogre, Dwarf, or Goblin.

$$\begin{aligned} & \forall x [\text{creator}(x) \Rightarrow (\text{ogre}(x) \vee \text{dwarf}(x) \vee \text{goblin}(x))] && \text{replace} \Rightarrow \\ & \forall x [\neg \text{creator}(x) \vee (\text{ogre}(x) \vee \text{dwarf}(x) \vee \text{goblin}(x))] && \text{drop } \forall \\ & \neg \text{creator}(x) \vee \text{ogre}(x) \vee \text{dwarf}(x) \vee \text{goblin}(x) \end{aligned}$$

2. Some creatures are monsters.

$$\begin{aligned} & \exists x [\text{creator}(x) \wedge \text{Monster}(x)] && \text{skolemization} \\ & \text{creator}(\bar{a}) \wedge \text{Monster}(\bar{a}) \end{aligned}$$

3. Monsters can eat some of the other creatures.

$$\begin{aligned} & \forall x [\text{Monster}(x) \Rightarrow \exists y \text{ creature}(y) \wedge \text{Eats}(x, y)] && \text{replace} \Rightarrow \\ & \forall x [\neg \text{Monster}(x) \vee \exists y \text{ creature}(y) \wedge \text{eats}(x, y)] && \text{sk} \\ & \forall x [\neg \text{Monster}(x) \vee \text{creature}(\text{sk}(x)) \wedge \text{eats}(x, \text{sk}(x))] && \text{drop } \forall \\ & \neg \text{Monster}(x) \vee \text{creature}(\text{sk}(x)) \wedge \text{eats}(x, \text{sk}(x)) && \text{distribute.} \\ & \neg \text{Monster}(x) \vee \text{creature}(\text{sk}(x)) \wedge \neg \text{Monster}(x) \vee \text{Eats}(\text{sk}(x), x) \end{aligned}$$

4. Creatures can defeat any creatures they eat.

$$\begin{aligned} & \forall x \forall y [\text{creature}(x) \wedge \text{Eats}(x, y) \Rightarrow \text{defeat}(x, y)] \\ & \forall x \forall y [\neg (\text{creature}(x) \wedge \text{eats}(x, y)) \vee \text{defeats}(x, y)] && \text{replace} \Rightarrow \\ & \forall x \forall y [\neg \text{creature}(x) \vee \neg \text{eats}(x, y) \vee \text{defeats}(x, y)] && \text{move } \neg \\ & \neg \text{creature}(x) \vee \neg \text{eats}(x, y) \vee \text{defeats}(x, y) && \text{drop } \forall \end{aligned}$$

5. Defeating is transitive. If x defeats creature y, and y defeats z, then x defeat z.

$$\begin{aligned} & \forall x \forall y \forall z [\text{defeat}(x, y) \wedge \text{defeat}(y, z) \Rightarrow \text{defeat}(x, z)] \\ & \neg (\text{defeat}(x, y) \wedge \text{defeat}(y, z)) \vee \text{defeat}(x, z) && \text{replace} \Rightarrow \\ & \neg \text{defeat}(x, y) \vee \neg \text{defeat}(y, z) \vee \text{defeat}(x, z) && \text{move } \neg \\ & \neg \text{defeat}(x, y) \vee \neg \text{defeat}(y, z) \vee \text{defeat}(x, z) && \text{drop } \forall \end{aligned}$$

6. If a creature is an Ogre, then that creature can eat dwarves.

$$\begin{aligned} & \forall x \forall y [\text{ogre}(x) \wedge \text{dwarf}(y) \Rightarrow \text{eat}(x, y)] \\ & \forall x \forall y [\neg (\text{ogre}(x) \wedge \text{dwarf}(y)) \vee \text{eat}(x, y)] && \text{replace} \Rightarrow \\ & \forall x \forall y [\neg \text{ogre}(x) \vee \neg \text{dwarf}(y) \vee \text{eat}(x, y)] && \text{move } \neg \\ & \neg \text{ogre}(x) \vee \neg \text{dwarf}(y) \vee \text{eat}(x, y) && \text{drop } \forall \end{aligned}$$

7. If a creature is a Dwarf, then that creature can defeat goblins.

$$\begin{aligned} & \forall x \forall y [\text{dwarf}(x) \wedge \text{goblin}(y) \Rightarrow \text{defeat}(x, y)] && \text{replace} \Rightarrow \\ & \neg (\text{dwarf}(x) \wedge \text{goblin}(y)) \vee \text{defeat}(x, y) && \text{move } \neg \\ & \neg \text{dwarf}(x) \vee \neg \text{goblin}(y) \vee \text{defeat}(x, y) && \text{drop } \forall \\ & \neg \text{dwarf}(x) \vee \neg \text{goblin}(y) \vee \text{defeat}(x, y) \end{aligned}$$

8. All goblins are monsters.

$$\begin{aligned} & \forall x [\text{goblin}(x) \Rightarrow \text{Monster}(x)] \\ & \forall x [\neg \text{goblin}(x) \vee \text{monster}(x)] && \text{replace} \Rightarrow \\ & \neg \text{goblin}(x) \vee \text{Monster}(x) && \text{drop } \forall \end{aligned}$$

Q4-3: sry ran out of time :/

Q4-4: sry ran out of time :/

Part 5. Liar Puzzle

#5 Determine who is telling the truth, proof by contradiction

suppose John is telling the truth F

John says "it was Nicole" T
Mark says it was not me T
but Mark must be lying X
John is a liar

Suppose Nicole is telling the truth F

Nicole says "it was Susan" T
Mark says it was not me" T
but mark is a liar X
Nicole is a liar

Suppose Mark is telling the truth F

Mark says "it was Not Me" T
John says "it was Nicole" F
Nicole says "it was Susan" F
Susan says "Nicole is a liar" T
but Susan must be a liar X
Mark is a liar

Suppose Susan is a liar F

we've proven John, Nicole, and Mark are liars T

from KB 1 person is telling the truth T

but Susan is also a liar X

Susan is telling the truth

Mark is a liar and says "it wasn't me"

Mark crashed the server

Q5-1: Susan

Q5-2 : Mark

Part 6. Bayesian Networks

Q6-1:0.06

Q6-2:0.06

Q6-3. $1 - 0.97 = 0.03$

Part 7. Bayesian Networks in Python

Q7-1:0.0305

Q7-2: 0.0337

Q7-3:0.1776

Q7-4: 0.3000

New application:

I see the most application opportunity for Bayesian Networks since very few real-world problems can be represented by only true and false values. One application of Bayesian Networks would be genetic prediction. Given a parent's genetics, you could determine the probabilities of the child's eye color, hair color, height, etc. You could also include environmental factors about the parents to gain even more insight, since some external factors can affect DNA. You could use this to scan for potential genetic conditions, like cancers or heart conditions, so people who are at higher risk know to get check-ups more often or avoid certain behaviors.

Feedback:

This assignment was tedious but necessary for developing a understanding of FOL and CNF. In particular, representing knowledge in FOL and converting it to CNF took practice to click. The problems that required us to prove statements were the most fun

Conclusions.

This assignment required a true understanding of FOL and CNF. We utilized these to represent and solve logical problems and answer questions about this. We learned how we can gain new knowledge from a knowledge base, and we began learning how to implement and use Bayesian Networks to represent probabilistic logical systems.